

Practical Quant: Stat Arb ที่ใช้ได้จริง

Part IV — Kalman ลงมือ: state-space, จูน Q/R, และรู้ว่ามันโกหกคุณตอนไหน

Part I ที่ทำไว้ว่า Kalman คือยอดบันได β — Part นี้ลงมือทำจริง: setup state-space, worked example ทีละ step, และคำถามที่สำคัญที่สุด — จะรู้ได้ยังไงว่า Kalman ที่คุณ tune มา "overfit" หรือ "ดีจริง"

 เส้นทางอ่าน

● **มือใหม่:** §4.1 ทบทวน + §4.2 state-space ภาษาคน + §4.3 worked example + สรุป ✓ · ● **มีพื้น:** เน้น §4.4 จุดยืน Kalman + §4.5 overfit diagnostic + §4.6 วินัยจน Q/R · **เริ่มต้น:** §4.5 คือสิ่งที่คอร์สทั่วไปไม่สอน

4.1 ทวนจาก Part I — ทำไมต้องมาถึงตรงนี้

อ่านเป็นภาษาคนก่อน

Part I ไต่บันได β มาแล้ว 3 ชั้น: OLS (คงที่ ง่ายสุด) $\rightarrow$ TLS (แก้ noise 2 ขา) $\rightarrow$ Rolling OLS (เดินตามเวลาแบบตัดหน้าต่าง) · ปัญหาของ Rolling คือ**ต้องเลือกขนาดหน้าต่าง** และที่ขอบหน้าต่าง β จะ "กระโดด" ทันทีที่ข้อมูลเก่าหลุดออกไป (ไม่ใช่ค่อย ๆ ลืม)

Kalman แก่ตรงนี้: แทนที่จะมี "หน้าต่าง" ตายตัว มันมี**สูตรถ่วงน้ำหนักที่ดีที่สุด**ในทางคณิตศาสตร์ระหว่าง "เชื่อ β เดิม" กับ "เชื่อราคาล่าสุด" — ค่อย ๆ ลืมอดีตแบบนุ่มนวล ไม่ใช่ตัดทิ้งดื้อ ๆ

4.2 State-space — สมการที่ต้องเข้าใจ (ไม่ใช่แค่ท่อง)

Kalman filter มองปัญหาเป็น 2 สมการ:

state (สิ่งที่มองไม่เห็น): $\beta_t = \beta_{t-1} + w_t$, $w_t \sim N(0, Q)$
 observation (สิ่งที่เห็น): $y_t = \beta_t \cdot x_t + v_t$, $v_t \sim N(0, R)$

 อ่านเป็นภาษาคนก่อน — 2 สมการนี้พูดว่าอะไร

- **สมการที่ 1 (state):** " β พรุ่งนี้ = β วันนี้ + การเปลี่ยนแปลงเล็กน้อยที่คาดเดาไม่ได้" — เหมือนบอกว่า "เชื่อว่า β ไม่กระโดดข้ามคืน แต่ก็ไม่นิ่งสนิท 100%"
- **สมการที่ 2 (observation):** "ราคาที่เราเห็นวันนี้ = β คูณราคาอีกขา บวก noise เล็กน้อย" — เหมือนบอกว่า "ราคาที่เราเห็นไม่ได้สะท้อน β เป๊ะ ๆ มันมี noise ปนมาด้วย"

Kalman ทำหน้าที่เดียว: ทุกวันที่มีราคาใหม่เข้ามา ชั่งน้ำหนักระหว่าง 2 อย่างนี้ แล้วปรับความเชื่อเรื่อง β

🔥 ทบทวนพื้นฐาน — Q และ R คืออะไร (ปุ่นที่สำคัญที่สุดในทั้ง Part)

Q (process noise / process variance)

ตอบคำถาม: "**β เปลี่ยนเร็วแค่ไหน**" — Q สูง = เชื่อว่า β ขยับได้เร็วและแรง (filter จะไวต่อการเปลี่ยนแปลง) · Q ต่ำ = เชื่อว่า β เกือบนิ่ง (filter จะเฉื่อย ไม่ค่อยขยับตาม)

R (observation noise / measurement variance)

ตอบคำถาม: "**เชื่อราคาล่าสุดแค่ไหน**" — R สูง = คิดว่าราคาที่เราเห็นมี noise เยอะ อย่าเพิ่งรีบเชื่อ (filter จะเฉื่อย) · R ต่ำ = คิดว่าราคาแม่นยำ เชื่อได้เต็มที่ (filter จะไว)

สังเกตไหมว่า Q กับ R "ดึงกันคนละทาง"?

Q สูง+R ต่ำ = ไวมาก (เกือบเหมือน rolling หน้าต่างสั้น) · Q ต่ำ+R สูง = เฉื่อยมาก (เกือบเหมือน β คงที่แบบ OLS) → **สิ่งที่สำคัญจริง ๆ ไม่ใช่ค่า Q, R แยกกัน แต่คืออัตราส่วน Q/R**
— นี่คือปุ่นเดียวที่ควบคุมพฤติกรรมทั้งหมดของ filter

🔧 เช็ความเข้าใจ

1. ถ้าตั้ง Q สูงมาก R ต่ำมาก — filter จะพฤติกรรมคล้ายวิธีไหนใน Part I? (คล้าย rolling OLS หน้าต่างสั้นมาก — ไวจัด กระจุกตามราคาทุกวัน)
2. ถ้าตั้ง $Q \approx 0$ — filter จะพฤติกรรมคล้ายวิธีไหน? (คล้าย OLS แบบคงที่ — เพราะบอกว่า β ไม่เปลี่ยนแปลง)

4.3 Worked example — เดิน Kalman ทีละวัน (มือไม่ฟังโลบรารี)

ก่อนเรียก pykalman ให้ดูว่าข้างในมันทำอะไรทีละสเต็ป — ใช้ตัวเลขสมมติเพื่อให้เห็นภาพ (ไม่ใช่ราคาจริง)

🚩 ตัวอย่างเดิน 1 วัน (สมมติ)

สมมติ ณ ต้นวัน: เชื่อว่า $\beta = 1.30$ (ความไม่แน่นอนของความเชื่อนี้ $P = 0.02$) · $Q = 0.001$ · $R = 0.05$ · วันนี้ราคา PEP (x) ขยับ +2.0 และราคา KO (y) ขยับ +2.9 จริง

ขั้น 1 — ทำนายล่วงหน้า (predict):

$$\beta_{\text{predict}} = \beta_{\text{เดิม}} = 1.30$$

$$P_{\text{predict}} = P_{\text{เดิม}} + Q = 0.02 + 0.001 = 0.021$$

ขั้น 2 — ทำนายว่าราคา y ควรเป็นเท่าไร:

$$y_{\text{predict}} = \beta_{\text{predict}} \times x = 1.30 \times 2.0 = 2.60$$

$$\text{innovation (ส่วนที่ทายผิด)} = y_{\text{จริง}} - y_{\text{predict}} = 2.9 - 2.60 = 0.30$$

ขั้น 3 — คำนวณ "น้ำหนักที่จะเชื่อ" (Kalman gain):

$$S = x^2 \times P_{\text{predict}} + R = 2.0^2 \times 0.021 + 0.05 = 0.134$$

$$K = x \times P_{\text{predict}} / S = 2.0 \times 0.021 / 0.134 \approx 0.313$$

ขั้น 4 — ปรับความเชื่อ β ใหม่:

$$\beta_{\text{ใหม่}} = \beta_{\text{predict}} + K \times \text{innovation} = 1.30 + 0.313 \times 0.30 \approx 1.394$$

$$P_{\text{ใหม่}} = (1 - K \times x) \times P_{\text{predict}} = (1 - 0.313 \times 2.0) \times 0.021 \approx 0.0079$$

สังเกต: β ขยับจาก 1.30 $\rightarrow$ 1.394 — **ไม่ใช่กระโดดไปเชื่อราคาใหม่เต็มที่ (จะได้ $\beta=1.45$ ถ้าเอา $2.9/2.0$) และไม่ใช่ไม่ขยับเลย** แต่ขยับตามสัดส่วนที่ K (Kalman gain) คำนวณให้ — นี่คือแก่นทั้งหมดของ Kalman: K คือน้ำหนักที่ปรับอัตโนมัติทุกวัน ตาม Q , R และความไม่แน่นอนสะสม

🧑‍🎓 อ่านเป็นภาษาคนก่อน — ทำไม P (ความไม่แน่นอน) ยิ่งเดินยิ่งเล็กลง

สังเกตว่า P ลดจาก 0.021 เหลือ 0.0079 — ยิ่งเห็นข้อมูลมาก ยิ่งมั่นใจใน β มากขึ้น (ความไม่แน่นอนลดลง) แต่ทุกวันที่ผ่านไป Q จะบวกความไม่แน่นอนกลับเข้ามานิดหนึ่งเสมอ (ขั้น 1) — เป็นสมดุลระหว่าง "มั่นใจขึ้นเพราะเห็นข้อมูล" กับ "ไม่มั่นใจเกินไปเพราะโลกเปลี่ยนได้ตลอด"

4.4 ★ จุดยืน Kalman — ชื่อสัตย์ ไม่เชียร์เกิน ไม่ปิดทั้ง

ตรงนี้เป็นสิ่งที่ต้องพูดให้ชัดก่อนไปต่อ เพราะ Part 0 เตือนไว้แล้วว่า "Kalman ไม่ใช่ edge"

แยกให้ขาด: กลไก vs การตั้งค่า

ตัว filter เอง ชื่อสัตย์เสมอ — มันใช้เฉพาะข้อมูลถึงวันนั้น (causal, ไม่ look-ahead) ถ้า Q/R ถูก fix ไว้ล่วงหน้า สัญญาณที่ได้ไม่มี lookahead bias เลย

ความ overfit ทั้งหมดที่จะเกิดขึ้น อยู่ที่ "การเลือก Q/R" ไม่ใช่ตัวกลไก — เหมือนมิดที่คมมาก ตัวมิดไม่ผิด อยู่ที่คนใช้จะบังคับมันแม่นยำแค่ไหน

✓ คุณค่าจริงของ Kalman

- คุณ drawdown ได้ดีกว่า rolling-OLS (ดูผลจริงในโน้ตบุ๊ก 01 — RMSE ต่ำกว่า rolling หลายสิบเท่าเมื่อ Q ตั้งถูก)
- ไม่กระตุกที่ขอบหน้าต่างเหมือน rolling
- ปรับ β ทุกวันแบบ optimal ตามทฤษฎี ถ้า assumption ถูก

⚠ สิ่งที่ Kalman ไม่ได้ให้ฟรี

- ไม่บอกว่า Q/R ที่ถูกต้องคืออะไร (§4.6 ต้องหาเอง)
- ไม่รู้ว่าความสัมพันธ์กำลังฟังจนกว่าจะสาย (โยง Part VII)
- ใครก็ตั้ง Kalman ได้ — ไม่ใช่ edge (Part 0)

"edge ไม่ได้อยู่ที่ 'ใช้ Kalman' — อยู่ที่วินัยการจูนมัน และรู้ว่าสมมติฐานฟังตอนไหน"

4.5 ★ จะรู้ได้อย่างไรว่า Kalman ที่คุณ tune มา "overfit"

นี่คือคำถามที่คอร์สทั่วไปข้าม — สอนแค่ "ใช้ Kalman" แต่ไม่สอนว่าจะรู้ได้อย่างไรว่ามันหลอกตัวเองอยู่ มี 4 วิธีตรวจ เรียงจากสำคัญที่สุด

① Q/R plateau vs peak ★★★ สำคัญสุด + ถูกสุด

👤 อ่านเป็นภาษาคนก่อน

ลอง Q หลาย ๆ ค่า (เช่น $1e-6$, $1e-5$, ..., $1e-1$) วัดผลบน out-of-sample แต่ละค่า แล้ว plot กราฟ — ถ้าผลลัพธ์ดีของจริง จะเห็น**"ที่ราบกว้าง"** (Q หลายค่าใกล้เคียงกันให้ผลดีพอ ๆ กัน) แต่ถ้าดีแค่ค่าเดียวแล้วพังทันทีที่ขยับนิดเดียว (**"ยอดแหลม"**) — นั่นคือสัญญาณอันตรายว่าคุณกำลัง overfit เข้ากับ noise ของช่วงทดสอบนั้นพอดี ไม่ใช่ค้นพบความจริง

โน้ตบุ๊ก 01 (S Kalman tuning) แสดงให้เห็นตรงนี้แล้ว: RMSE ที่ดีมากเกิดขึ้นเมื่อ Q ใกล้เคียงอัตราการเปลี่ยนจริงของ β — ลองเปลี่ยน Q ให้ห่างจากนั้น (เช่น 100 เท่า หรือ $1/100$ เท่า) แล้วดูว่า RMSE แยกแบบ "ค่อย ๆ" หรือ "ร่วงหน้าผา"

② Innovation whiteness — ตัดสินจาก "ทำนายแม่นยำแค่ไหน" ไม่ใช่จาก PnL

👤 อ่านเป็นภาษาคนก่อน

Innovation คือ "ส่วนที่ filter ทายผิด" (ดูขั้น 2 ใน worked example) — ถ้า filter ตั้งค่าถูก innovation ควรมีลักษณะ**สุ่มล้วน ๆ** (ไม่มีรูปแบบซ้ำ ไม่ autocorrelate) และขนาดของมันควรตรงกับที่ filter ทำนายไว้เอง (ค่า S ในขั้น 3 ของ worked example)

ถ้า innovation ยังมีรูปแบบ (เช่น ทายผิดทางเดิมติดกันหลายวัน) = filter ยัง**ตามไม่ทัน** Q อาจต้องสูงขึ้น

★ ทำไมข้อนี้สำคัญ: มันตัดสิน filter โดยไม่ต้องแตะ PnL เลย

นี่คือกุญแจที่ป้องกัน overfit ได้ตรงจุดที่สุด: ถ้าคุณดู Q/R จาก PnL ที่ backtest ออกมาสวย คุณกำลัง fit noise ของช่วงนั้นเข้าไปในพารามิเตอร์ แต่ถ้าคุณดูจาก "innovation ทำนายแม่นยำไหม" (ซึ่งเป็นคุณสมบัติทางสถิติล้วน ๆ ไม่เกี่ยวกับกำไรขาดทุน) คุณกำลังเชื่อว่าโมเดลอธิบายข้อมูลได้ถูกต้อง ไม่ใช่เชื่อว่าบังเอิญทำกำไรงวดนี้

③ + ④ อีก 2 diagnostic (สรุปสั้น)

- **IS vs OOS gap:** ถ้า in-sample ดีมากแต่ out-of-sample แย่ลงฮวบ = overfit ชัดเจน (โย่ง deflated Sharpe, walk-forward — Part IX)
- **Q/R stability ข้าม sub-period:** ถ้าจน Q/R บนปี 2023 ได้ค่าหนึ่ง แต่จนบนปี 2024 ได้ค่าที่ต่างลิบลิบ — Q/R "ที่ดีที่สุด" ไม่ใช่คุณสมบัติที่นิ่งจริง แปลว่าคุณกำลังจับ noise เฉพาะช่วง ไม่ใช่ค้นพบพารามิเตอร์ที่แท้จริง

4.6 จูน Q/R ไม่ให้หลอกตัวเอง — ลำดับวินัย

⚠ กับดักที่ tutorial ทัวไปทำ (และทำไมมันหลอกตัวเอง)

วิธีที่พบบ่อยสุดบนอินเทอร์เน็ต: grid-search Q/R หลาย ๆ ค่า เลือกค่าที่ให้ **PnL backtest สูงสุด** — ฟังดูสมเหตุสมผล แต่นี่คือการ **overfit ตรง ๆ**: คุณกำลังเลือกพารามิเตอร์ที่เข้ากับ noise ของอดีตพอดี ไม่ต่างจากการ curve-fit เส้นให้ผ่านทุกจุดในข้อมูล historical

ลำดับที่ควรทำจริง (จากดีที่สุดไปหา fallback):

ลำดับ 1 — ประมาณเชิงสถิติ ไม่ใช่ตาม PnL ★★★

ใช้ **Maximum Likelihood (MLE) / Expectation-Maximization (EM)** — วิธีนี้หา Q/R ที่ทำให้ข้อมูลราคาที่เกิดขึ้นได้ "สมเหตุสมผลที่สุด" ทางสถิติ (maximize likelihood) ไม่ใช่หาที่ให้กำไรเยอะสุด — ในโค้ดจริงมักเรียกเป็นเมธอดเดียว เช่น pykalman มีฟังก์ชัน `.em()` ให้ใช้ได้ตรง ๆ

⚠ caveat

EM หลุดไปที่ค่า degenerate ได้ (เช่น $R \rightarrow 0$ ซึ่งไม่สมเหตุสมผล) — ต้องใส่ขอบเขต (bound) กันไว้เสมอ

ลำดับ 2 — ผูกกับความหมายเชิงเศรษฐกิจ

แทนที่จะเอา Q/R เป็นตัวเลขลอย ๆ ให้ผูกกับสิ่งที่มีความหมาย:

- $R \approx$ ความแปรปรวนจาก microstructure/bid-ask ของ spread (มีความหมายจริง วัดได้จากข้อมูล)
- $Q \leftrightarrow$ "เชื่อว่า β ควรเปลี่ยนแบบมีนัยสำคัญทุก ๆ ~กี่เดือน" — แปลงความเชื่อนี้เป็นตัวเลข Q ได้
- อัตราส่วน $\lambda=Q/R \leftrightarrow$ "effective window" ที่เทียบเคียงได้กับ EWMA span — **เลือก lookback ด้วยความเชื่อเกี่ยวกับธุรกิจ ไม่ใช่ลองแล้วดูว่า backtest สวยไหม**

ลำดับ 3 — ถ้าจำเป็นต้อง search จริง ๆ: train/lock/test

fit บน train set $\rightarrow$ **lock ค่าไว้ ห้ามแก้อีก** $\rightarrow$ วัดผลบน OOS ที่ไม่เคยแตะ · ใช้ purged/embargoed cross-validation (Part IX) · ถ้าลองหลายค่า ต้องหัก deflated Sharpe ตามจำนวนที่ลอง (เหมือนปัญหา multiple-testing ใน Part II)

ลำดับ 4 — วินัยเสริม: เลือกที่ราบ + ปุ่มเดียว

เลือกจุดกลางของ "ที่ราบ" ใน §4.5① ไม่ใช่จุดยอด · ลด parameter ที่ต้องจูนให้เหลือน้อยที่สุด — เช่น fix R จากการวัด microstructure noise จริง แล้วจูนแค่ Q ตัวเดียว (หรือจูนแค่ $\lambda=Q/R$ ตัวเดียว) ยิ่งปุ่มน้อย ยิ่ง overfit ยาก

4.7 สรุป Part IV + สะพานสู่ Part V

✓ 3 อย่างที่ต้องจำจาก Part IV

1. **Q/R คือปุมเดียวที่สำคัญสุด** — Q สูง/R ต่ำ = ไว (เหมือน rolling สั่น) · Q ต่ำ/R สูง = เนื่อย (เหมือน OLS คงที่)
2. **ตัดสินใจ filter จาก innovation ไม่ใช่จาก PnL** — จูนตาม PnL = overfit ตรง ๆ; จูนตามสถิติ (MLE/EM) หรือความหมายเชิงเศรษฐกิจ = ปลอดภัยกว่ามาก
3. **Q/R plateau vs peak** คือวิธีเช็ค overfit ที่ถูกและตรงที่สุด — ที่ราบกว้าง = เชื่อได้ · ยอดแหลม = หลุด

สมมติฐานหลักที่ Part นี้ใช้มาตลอดคือ **β เดินแบบ random walk** (ไม่มีบ้าน เดินไปเรื่อย ๆ ไม่จำกัด) — แต่คู่ที่ cointegrate กันจริง ๆ ควรมี β ที่ "นิ่ง" ในระดับหนึ่ง ไม่ใช่ล่องลอยไม่มีจุดหมาย และมีอันตรายที่ลึกกว่านั้น: ถ้า Kalman ปรับ β ไวกเกินไป มันอาจลื่นสัญญาณที่คุณอยากเทรดเข้าไปในตัว β เองจนไม่เหลืออะไรให้เทรด

"ถ้า β ปรับตัวไวจนตามทันทุกอย่าง — spread จะดู 'ปกติ' ตลอดเวลา แล้วคุณจะเทรดอะไร?"

📖 ศัพท์ใหม่ในตอนนี้

- **State-space model** — กรอบที่แยก "สิ่งที่มองไม่เห็น" (state, เช่น β) ออกจาก "สิ่งที่สังเกตได้" (observation, เช่น ราคา)
- **Q (process noise)** — ความไม่แน่นอนของการเปลี่ยนแปลง state ต่อวัน (" β เปลี่ยนเร็วแค่ไหน")
- **R (observation noise)** — ความไม่แน่นอนของการสังเกต ("เชื่อราคาล่าสุดแค่ไหน")
- **Kalman gain (K)** — น้ำหนักที่ filter ใช้ผสมความเชื่อเดิมกับข้อมูลใหม่ ปรับอัตโนมัติทุกวัน
- **Innovation** — ส่วนที่ filter ทายผิดในแต่ละวัน; ใช้ตรวจว่าโมเดลถูกหรือไม่ (ไม่ใช่ดูจาก PnL)
- **MLE / EM** — วิธีประมาณ Q/R จากสถิติของข้อมูล แทนการลองผิดลองถูกจาก PnL

อ้างอิงตอนนี้

- Palomar, D.P. (2025) *Portfolio Optimization: Theory and Application*, Cambridge — §15.6 Kalman filtering for pairs trading
- Bar-Shalom, Y., Li, X.R. & Kirubarajan, T. — innovation consistency / NIS test (มาตรฐานของสาย estimation theory)
- Chan, E. (2013) *Algorithmic Trading* — โฉด Kalman pairs ตัวอย่าง

 **ลงมือ:** notebook vol2-code/01_beta_ladder.ipynb (ดูส่วน Kalman + ค่าเตือนเรื่อง Q ที่ตั้งไว้ "บังเอิญถูก") — notebook 04 (Part V) จะลงลึก Q/R tuning แบบเต็ม

↔ เชื่อมเล่ม 1: arb-part5 §19.2 (Kalman concept) · Part I (บันได β , super-consistency)